RubberDuck: Intent-Aware Detection of Subtle Logic Bugs

AI-Assisted Software Engineering: Final Project Report

Nvafeomo
Penn State Great Valley
Repository: https://github.com/Nvafeomo/Rubber-Duck

October 2026

Abstract

RubberDuck is a Git pre-commit hook that asks the developer what a change is supposed to do and then checks the staged Python code against that answer. It targets three kinds of bugs that run without errors but still do the wrong thing: using the wrong variable, reading the wrong dictionary key, and looping over the wrong collection. Each review is a single Gemini call that sees the staged diff, the full file, the developer's intent, and definitions pulled from other files with `jedi`. Instead of declaring code wrong, it asks up to three questions tied to specific lines, and the developer can abort the commit or dismiss the questions. On 15 bugs seeded into the TinyDB project, RubberDuck caught 14 in every configuration, including the one without intent or context, so that test did not show a benefit from intent. A second benchmark of 12 hand-written cases, where the intent is the only way to tell that the code is wrong, showed a clear difference: with the intent RubberDuck caught all 12 bugs with one false alarm, and without it caught 5 with four false alarms.

1 Problem and Motivation

Modern IDEs and static analyzers reliably catch syntax errors and common bugs such as type mismatches and null dereferences, but they can't catch code that is valid but inconsistent with what the developer meant. Code can type-check and pass standard analysis and still do the wrong thing. Examples include indexing the wrong dictionary key, iterating over the wrong list, and reusing the wrong variable name.

I focused on individual developers and small teams rather than larger teams with QA-gated releases. Solo developers and small teams usually don't have dedicated code reviewers and often work across multiple files, which makes it easy to introduce subtle errors and miss them during a quick look over the code.

This matters because fixing these issues is usually trivial, but finding them can be very expensive depending on when they are noticed. If they show up far downstream in testing or production, they cost more and are harder to trace back to their source. In a large codebase, fixing one of these bugs can mean manually tracing variable uses and control flow for hours, all for what turns out to be a one-line fix. My goal was to catch these bugs early, at the moment of the commit, by having an LLM review the staged change against the developer's stated intent and related code from the rest of the project.

2 AI4SE / SE4AI Context

This project is primarily **AI4SE**. It applies an LLM to the code review and debugging activity during active development. Instead of fully automating bug detection, RubberDuck supports the developer's judgment by asking targeted questions about possible inconsistencies. Because it sits in the pre-commit hook, developers can catch and fix subtle bugs right before they commit, without a separate review process. The LLM supports the review instead of replacing it: it checks the code against what the developer meant to do, and the developer makes the final call.

The project also has a secondary **SE4AI** side. RubberDuck is itself a software system that had to be built, tested, and evaluated like any other. The parts that gather context, build the prompt, parse the model's answer, handle a busy API, seed test bugs, and measure accuracy are all regular code with their own tests, continuous integration, and design decisions. Only one step in the pipeline actually uses the model.

3 Current Practice and Existing Approaches

Static analyzers and linters catch rule-based issues such as style problems, type errors, and unused code, but they have no notion of developer intent. A loop over the wrong list is perfectly valid to a linter.

DeepBugs [1] is the closest prior work. It trains a model on artificially created bugs to detect name-based mistakes like swapped arguments. It predates LLMs and can't take a task description into account.

GitHub Copilot's debugging features explain and fix errors, crashes, and failing tests. They usually start from a symptom. The bugs RubberDuck targets don't produce one: the code runs cleanly and just gives the wrong answer, so there is nothing to start debugging from.

AI code review tools like Copilot code review [3] and the CodeRabbit CLI [4] now review pull requests and, in some setups, local uncommitted changes. They run when the developer asks for a review, and they work from the code and repository-wide instructions rather than a per-change statement of what the change was supposed to do. They are also broader tools: they check for security, performance, and style issues and work in many languages, which RubberDuck does not.

RubberDuck fills a narrower gap. It runs automatically on every commit, asks for the intent of that specific change while it is still fresh, and only looks for three bug types.

The design also borrows from **rubber duck debugging** [2], where a developer explains their code out loud, often to an object on their desk, and catches their own mistakes just by saying their reasoning step by step. RubberDuck turns this habit into an automatic tool, swapping the silent object for one that can actually see the code and ask a real question back.

4 Proposed Solution and Requirements

RubberDuck is a pre-commit hook that reviews staged Python changes before they are committed. When the developer runs `git commit`, it:

1. reads the staged diff to find which lines changed,
2. pulls the full staged content of each file so the model sees the whole file,
3. asks the developer "what should this change do?",

4. finds definitions in other project files for names used on the changed lines, and
5. sends all of this to Gemini in one request, which returns either no concerns or up to three questions tied to specific lines.

If there are questions, the commit is blocked until the developer either aborts to fix the code or dismisses the questions and commits anyway. The hook can also be skipped with `git commit --no-verify` or forced with `RUBBERDUCK_FORCE=1`.

The full requirements and acceptance criteria are in `docs/requirements.md` in the repository. The main ones are summarized in Table 1.

ID	Requirement	Checked by
FR1	Runs automatically from a Git pre-commit hook on staged `.py` files	Live demo
FR2	Asks for the intent before reviewing	Live demo, review tests
FR3	Sends diff, full file (or enclosing function for files over 400 lines), intent, and cross-file definitions	Source view and context tests
FR4	One model call, at most three concerns, each with file, line, and question	Parsing tests
FR5	Concerns block the commit unless dismissed or forced	Review tests, live demo
FR6	Retries and falls back to other models when Gemini is busy	Retry tests
NFR1	Fast enough to run on every commit	Latency in evaluation
NFR2	API key never committed; tests need no key	CI

Table 1: Main requirements and how each one is checked.

Out of scope: languages other than Python, an editor plugin or other UI, bug types other than the three above, and letting the model search the codebase on its own.

5 Architecture and Implementation

RubberDuck is a small Python package with a command line entry point. The live tool is the `rubberduck review` command, which the pre-commit hook runs. Figure 1 shows the flow.

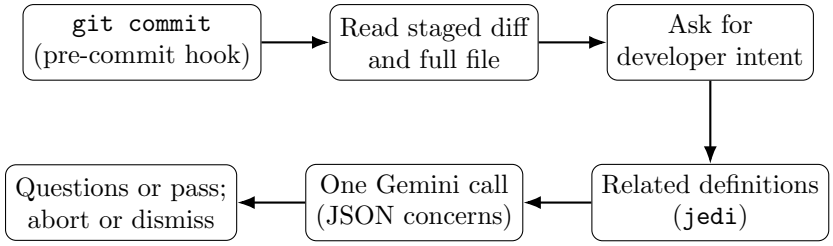


Figure 1: RubberDuck review workflow.

5.1 Components

- **Trigger:** a plain Git pre-commit hook that runs `rubberduck review`.
- **Staged code:** `git diff --cached` for changed line numbers and `git show :path` for the full staged file.

- **Context retrieval:** Python `ast` to find names on changed lines and `jedi` [6] to find where they are defined.
- **LLM:** Google Gemini [5], `gemini-3.5-flash-lite` by default, temperature 0, with a JSON response schema.
- **Bug seeding:** three custom mutation operators in `mutations.py`.
- **Evaluation corpus:** TinyDB [7], cloned at run time.
- **Testing and tooling:** pytest, pandas, ruff, GitHub Actions, Docker.

5.2 Design decisions

The main decisions are recorded as Architecture Decision Records in `docs/adr/`.

Whole file, not just the diff. The hook only uses the diff to figure out which lines changed. The code sent to the model is the whole staged file, so it sees more than the few lines a normal diff shows around a change. Files over 400 lines are reduced to the function or class that contains the change, since sending a huge file on every commit would cost more and slow things down without making the results better.

jedi for cross-file context. To pull in code from other files, I used `jedi`, the same library that powers "go to definition" in many Python editors, instead of writing a symbol lookup from scratch. When a changed line uses a name defined somewhere else in the project, that definition is added to the prompt (up to six snippets of 40 lines each). This works well for plain names and imports. It does not always work for method calls where the object's type can't be figured out just by reading the code, and that limitation was accepted for the proof of concept.

One model call, not an agent. Each file gets a single request with the rules, the intent, the diff, the numbered file, and the related definitions, instead of a multi-step agent that decides on its own to go look up more information. A single request was simpler to build and test in seven weeks, it is fast enough to run on every commit, and every run gets the exact same inputs, which matters for the evaluation. An agent version is listed as future work.

Questions, not verdicts. The model only sees part of the codebase and a one-line intent, so it will sometimes be wrong. The prompt tells it to phrase each concern as a short question about one specific line, to return at most three, to return nothing if the change fits the intent, and to stay away from style, type hints, and design opinions.

Handling a busy API. During testing the free tier of Gemini often returned overload and rate limit errors. RubberDuck retries once after a short wait and then tries any fallback models listed in `RUBBERDUCK_FALLBACK_MODELS` before giving up with a clear message.

5.3 Software engineering practices

The repository includes 34 automated tests that use fake model responses and temporary Git repositories, so they run without an API key. A GitHub Actions workflow runs a ruff lint check and the tests on Python 3.10 and 3.12 on every push, then builds the Docker image and runs the tests inside it. A Dockerfile and `docker-compose.yml` provide services for the tests and both evaluations. The backlog is tracked as GitHub issues grouped into milestones, and the requirements, architecture, and decision records are in `docs/`. The README explains how to set up, run, test, and reproduce the evaluation.

6 Proof of Concept

This section walks through the working proof of concept, from the code behind the review to a full run of the live demo. The demo uses an empty Git repository (`demo/`) created by a reset script in `demo-tools/`, so each run starts clean.

6.1 The code behind the review

Listing 1 shows the instructions that every review request starts with. They spell out the three bug types, tell the model to return nothing when the change fits the intent, and require each concern to be a question about one numbered line. This is the part of the system that keeps the output short and avoids generic feedback.

```
INSTRUCTIONS = """You are RubberDuck, a pre-commit reviewer for subtle logic bugs.

The code under review is syntactically valid. Ask a question only when a changed line is
    inconsistent with the developer intent or with a related definition. The inconsistencies
    that matter are:
- a wrong variable
- a wrong dictionary key
- a loop iterating over the wrong collection

Rules:
- If the change fits the intent, return an empty concerns list.
- Do not declare code wrong. Phrase each concern as a short question the developer can confirm
    or dismiss.
- Cite the file path given below and the exact line number from the left margin of the numbered
    source. Ignore numbers in the diff.
- Ask about one specific line. Return at most three concerns.
- Do not comment on style, formatting, type hints, missing tests, or speculative design.
"""
```

Listing 1: The review instructions in `rubberduck/prompt.py`.

Listing 2 shows the core of how cross-file context is gathered. For every name on a changed line, `jedi` is asked where it is defined, the same way an editor's "go to definition" works. Definitions in the same file are skipped since the model already has that file, and anything outside the repository, including the standard library, is skipped by the `relative_to` check. Error handling and a few bookkeeping lines are left out.

6.2 Live demo walkthrough

For the demo, an AI coding agent wrote a short weekly payroll program with three intentional bugs, one of each type, using the prompt in `demo-tools/GENERATE_DEMO_PROMPT.md`. The program runs without errors and prints a reasonable looking payroll report. Most of the file is sample data for employees and contractors, so Listing 3 shows only the three functions where the bugs are, with their real line numbers:

- **Line 74, wrong dictionary key:** overtime pay is calculated from `regular_hours` instead of `overtime_hours`.
- **Line 86, wrong variable:** `take_home_pay` computes `take_home` but returns `gross`.
- **Line 91, wrong loop collection:** the payroll by department loops over the `contractors` list instead of `workers`.

```
project = jedi.Project(str(repo))
script = jedi.Script(code=source, path=str(file_path), project=project)

for line, column in targets:          # names used on the changed lines
    definitions = script.goto(line, column, follow_imports=True)
    for definition in definitions:
        resolved = Path(definition.module_path).resolve()
        if resolved == origin:        # same file: already in the prompt
            continue
        try:
            relpath = resolved.relative_to(repo_resolved).as_posix()
        except ValueError:            # outside the repo, e.g. the standard library
            continue
        numbered = _definition_text(resolved, int(definition.line), settings.max_snippet_lines)
        snippets.append(Snippet(relpath=relpath, line=int(definition.line),
                                description=str(definition.description), numbered=numbered))
        if len(snippets) >= settings.max_snippets:
            return snippets
```

Listing 2: Cross-file context with `jedi`, from `related_snippets` in `rubberduck/context.py` (shortened).

```
def weekly_gross(worker):
    regular_pay = worker["regular_hours"] * worker["hourly_rate"]
    overtime_pay = (
        worker["regular_hours"]
        * worker["hourly_rate"]
        * worker["overtime_multiplier"]
    )
    return regular_pay + overtime_pay


def take_home_pay(worker):
    gross = weekly_gross(worker)
    withholding = gross * worker["withholding_rate"]
    benefits = worker["benefit_deduction"]
    take_home = gross - withholding - benefits
    return gross


def department_payroll(workers, contractors):
    totals = defaultdict(float)
    for worker in contractors:
        totals[worker["department"]] += weekly_gross(worker)
    return totals
```

Listing 3: The three bugged functions in the AI-generated demo program (`demo/payroll.py`, lines 71–93).

After `git add .`, running `git commit` triggers the hook. RubberDuck asks what the change should do, and I typed the same intent the agent was given. It then lists its questions, each tied to a line, and asks whether to abort or dismiss (Figure 2). All three questions point at the exact bugged lines from Listing 3 and suggest the intended field, variable, or list. Pressing `a` blocks the commit.

```
(.venv) PS C:\Users\alhaj\OneDrive\Desktop\Nvafeomo - CS\Rubber-Duck> cd ./demo
(.venv) PS C:\Users\alhaj\OneDrive\Desktop\Nvafeomo - CS\Rubber-Duck\demo> git add .
>> git commit -m "Add weekly payroll report"
RubberDuck: what should this change do?
> Calculate each employee's weekly gross pay from regular and overtime hours, their take-home pay after withholding and benefits, and the employee payroll total for each department.

RubberDuck:
1. payroll.py:74
   Should the overtime pay calculation use overtime_hours instead of regular_hours?
2. payroll.py:86
   Should take_home_pay return the take_home variable instead of gross?
3. payroll.py:91
   Should the loop iterate over workers instead of contractors to calculate department payroll?
[a]bort  [d]ismiss and commit:
```

Figure 2: The pre-commit hook asking for the intent, then asking about lines 74, 86, and 91 before letting the developer abort or dismiss.

After fixing the three lines (`overtime_hours` on line 74, `return take_home` on line 86, and looping over `workers` on line 91), the same commit with the same intent comes back with no concerns and goes through (Figure 3).

```
(.venv) PS C:\Users\alhaj\OneDrive\Desktop\Nvafeomo - CS\Rubber-Duck\demo> git add .
>> git commit -m "Add weekly payroll report"
RubberDuck: what should this change do?
> Calculate each employee's weekly gross pay from regular and overtime hours, their take-home pay after withholding and benefits, and the employee payroll total for each department.

RubberDuck: no concerns in 1 file.
[master 2403768] Add weekly payroll report
 1 file changed, 94 insertions(+)
 create mode 100644 payroll.py
(.venv) PS C:\Users\alhaj\OneDrive\Desktop\Nvafeomo - CS\Rubber-Duck\demo>
```

Figure 3: A clean review after the fixes, letting the commit through.

6.3 Testing and automation

All 34 automated tests pass locally and in GitHub Actions. Figure 4 shows a CI run where the tests passed on Python 3.10 and 3.12, followed by the Docker build, which runs the same tests inside the container.

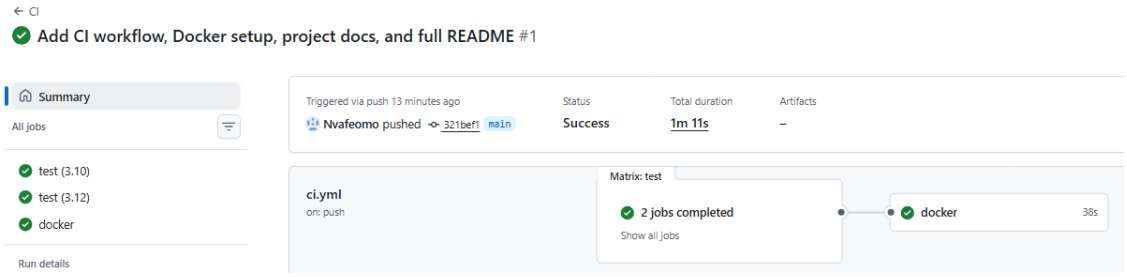


Figure 4: Continuous integration on GitHub Actions: tests on Python 3.10 and 3.12, then the Docker build.

7 Evaluation Method

Evaluation question: Does RubberDuck with the developer's intent and cross-file context detect these bugs more accurately than the same model call without them?

I ran two evaluations. Both call the same review code as the live tool and both are in the repository with instructions to reproduce them.

7.1 Seeded bugs in TinyDB

The first evaluation uses functions from TinyDB, a real, medium-sized open-source Python project. I picked a less famous project partly because the model may have memorized a very popular codebase. The three mutation operators each change one line of a function: swapping a variable for another in-scope variable, swapping a dictionary key for another key used in the same function, and swapping the collection a loop runs over. I originally planned to use MutPy [8], but it doesn't have operators for these bug types, so I wrote them myself.

I seeded 5 bugs of each type (15 total) and also reviewed the same 15 functions without any change, to measure false alarms. Functions with docstrings were picked first, because the docstring stands in for the developer's intent. Each buggy and clean function was reviewed under four conditions:

- **Full:** intent and cross-file context
- **Context only:** no intent
- **Intent only:** no cross-file context
- **Code only:** neither, which works as the baseline

That is 30 reviews per condition and 120 model calls in total.

Defining a hit: a flag only counts as correct if it points to the exact line that was changed to create the bug, not just any concern somewhere in the function. This keeps the results from looking better than they really are because of a vague flag.

Metrics: recall (seeded bugs caught), precision (correct flags out of all flags raised, including any on clean code), false-positive rate (clean functions that got flagged), and average latency per review.

7.2 Intent benchmark

The TinyDB results (Section 8) showed no difference between conditions, likely because a swapped variable in a library function often looks wrong even without knowing the intent. To test the cases where intent should actually matter, I wrote 12 small programs (`evaluation/intent_cases.py`), such as weekly payroll, course grades, and library late fees. In each one the buggy version is perfectly reasonable code on its own, for example paying employees by scheduled hours instead of worked hours, and the only way to know it is wrong is the stated intent. Each case is reviewed buggy and clean, with and without the intent, as a brand new file (48 model calls). The hit definition and metrics are the same as above.

8 Results and Analysis

8.1 TinyDB

Table 2 shows the results of the seeded-bug evaluation.

Condition	Recall	Precision	False alarms	Avg latency
Full (intent + context)	14/15 (0.93)	0.88	2/15 (0.13)	0.92 s
Context only	14/15 (0.93)	0.93	1/15 (0.07)	0.93 s
Intent only	14/15 (0.93)	0.93	1/15 (0.07)	0.99 s
Code only (baseline)	14/15 (0.93)	0.88	2/15 (0.13)	0.88 s

Table 2: Seeded-bug results on TinyDB (15 bugs, 15 clean functions, per condition).

RubberDuck caught 14 of 15 bugs in every condition. By bug type the results were the same in all four conditions: 5 of 5 wrong variables, 5 of 5 wrong loop collections, and 4 of 5 wrong dictionary keys. The one miss was the same dictionary key swap every time. The differences in precision come down to a single false alarm, so on these bugs intent and context made no measurable difference. Reviews took about a second each, which is fast enough to run on every commit.

The honest reading is that this test was too easy to show the effect I was looking for. With only 15 bugs, and bugs that often look wrong on their own, even the code-only baseline did well. These are the results I presented in class, and they are the reason I added the intent benchmark.

8.2 Intent benchmark

Table 3 shows the results on the 12 intent-dependent cases.

Condition	Recall	Precision	False alarms	Avg latency
With intent	12/12 (1.00)	0.92	1/12 (0.08)	0.96 s
Code only	5/12 (0.42)	0.42	4/12 (0.33)	0.77 s

Table 3: Results on the intent benchmark (12 buggy and 12 clean programs per condition).

Here the intent made a large difference. With the intent, RubberDuck caught every bug on the exact line, and its only wrong flag was a single false alarm on one of the clean programs. Without the intent it caught 5 of 12, and it flagged 4 of the 12 clean programs, so less than half of its flags were correct. The gap showed up for every bug type:

Bug type	With intent	Code only
Wrong dictionary key	4/4	2/4
Wrong loop collection	4/4	1/4
Wrong variable	4/4	2/4

Table 4: Bugs caught by type on the intent benchmark.

Figure 5 shows the summary printed at the end of the run.

```
By condition
condition      caught  precision  clean flagged  avg latency
with-intent     12/12       0.92           1/12       0.96 s
code-only        5/12       0.42           4/12       0.77 s

Bugs caught by type
bug type                 with-intent    code-only
wrong_dict_key                   4/4          2/4
wrong_loop_collection            4/4          1/4
wrong_variable                   4/4          2/4

Wrote 48 row(s) to results\intent-eval-results.csv
```

Figure 5: Summary output of the intent benchmark.

This makes sense given how the cases were built. Without the intent, paying employees by scheduled hours or counting every student on the roster looks like a reasonable choice, so the model either misses the bug or starts guessing and flags code that is actually fine. With one sentence saying what the code is supposed to do, the mismatch becomes obvious. The cost was small: reviews with the intent took about 0.2 seconds longer on average because of the extra text in the prompt.

8.3 Comparing the two evaluations

The two evaluations seem to disagree. The TinyDB run, which is the one I presented in class, showed no benefit from intent, while the intent benchmark, which I ran after the presentation, showed a large one. Table 5 puts the two side by side, comparing the full system to the code-only baseline in each.

	TinyDB (seeded bugs)	**Intent benchmark**
Code under review	Real library functions	12 short programs I wrote
How bugs were made	One-line mutation, chosen automatically	One-line change, written by hand so both versions look reasonable
Where the intent came from	The function's docstring	One sentence written for the case
Conditions	Full, context only, intent only, code only	With intent, code only
Size	15 buggy + 15 clean	12 buggy + 12 clean
Recall (full vs. code only)	14/15 vs. 14/15	12/12 vs. 5/12
False alarms (full vs. code only)	2/15 vs. 2/15	1/12 vs. 4/12
Avg latency (full vs. code only)	0.92 s vs. 0.88 s	0.96 s vs. 0.77 s

Table 5: The two evaluations side by side.

I think the difference comes down to three things.

What the bugs look like. The TinyDB mutations swap a variable, key, or loop target for another name in the same function, picked automatically. The result often looks out of place even without

knowing the goal, for example a loop over a variable that has nothing to do with the rest of the function, so the model can catch it from the code alone. In the intent benchmark, both versions are normal code. Paying employees by `scheduled_hours` is just as reasonable as paying them by `worked_hours` until you know which one was meant.

How good the intent is. In TinyDB the intent was the function's docstring, which often describes the function in general terms and doesn't say anything about the specific line that changed. In the intent benchmark the intent is one sentence that says exactly what the code should do, which is closer to what a developer would type at commit time.

Size and setup. TinyDB had all four conditions, so it also tested cross-file context, which the intent benchmark did not (each case is a single file). Both samples are small, so a single bug or false alarm moves the numbers noticeably, and the precision differences in TinyDB come down to one flag.

Together, the two evaluations suggest that intent matters most for exactly the bugs RubberDuck was designed for, the ones where the code is reasonable on its own and is only wrong compared to what the developer meant. For bugs that already look suspicious, like a swapped variable in a library function, the model does fine without it. The intent benchmark is the closer match to the live demo and to how the tool would be used, but it is also a set of cases I wrote myself with the intent in mind, so it likely shows the best case. Neither evaluation showed a measurable effect from cross-file context on its own, which is something a larger test with more multi-file changes would need to look at.

8.4 Quality of the questions

Precision and recall don't show whether a correct flag was actually useful. Looking through the questions by hand, the correct ones pointed at the right field or variable by name, for example asking whether hours should come from `worked_hours` instead of `scheduled_hours` so employees are paid for the hours they actually worked. That kind of question is quick to confirm or dismiss, which is the point of asking instead of declaring the code wrong.

9 Risks, Limitations, and Lessons Learned

Limitations.

- Python only, and terminal only. Committing from an editor's Commit button skips the intent prompt.
- Only three bug types. RubberDuck is not a general code reviewer.
- The questions are only as good as the intent the developer types.
- `jedi` can't always resolve method calls on objects whose type isn't obvious.
- The TinyDB sample is small (15 bugs), seeded bugs may be easier to spot than real ones, and docstrings are a weaker signal than a developer's own intent.
- The free tier of Gemini was often busy. Retries and fallback models helped, but a review can still fail, in which case `--pass-on-error` lets the commit through.

Risks. The model can be wrong in both directions. A missed bug gives false confidence, and repeated false alarms would lead a developer to dismiss everything or turn the hook off. Keeping the output to a few specific questions is meant to reduce that second risk. The code is also sent to an external API, which may not be acceptable for private codebases.

Lessons learned.

- The evaluation has to be hard enough to separate the conditions. My first benchmark mostly showed that these seeded bugs are easy to spot. Writing cases where the intent actually matters is what finally showed the effect I was looking for.
- Planning for an unreliable API early would have saved time. Retries and fallback models were added after free-tier overloads interrupted evaluation runs and demo rehearsals.
- Keeping the model call to one step made testing much easier. Everything except the call itself could be tested with fake responses, so the tests run in CI without a key.
- Using an AI agent to write the demo program, and RubberDuck to check it against the agent's own instructions, turned out to be a good fit for the tool and a natural use case for it going forward.

Future work. Letting the model ask for more definitions instead of one fixed prompt, supporting more bug types and languages, a larger evaluation with real bugs from project histories, and an editor plugin.

Use of AI tools. AI coding assistants were used to help write parts of the code, tests, documentation, and demo materials. All of it was reviewed, run, and tested by me.

References

[1] M. Pradel and K. Sen, *DeepBugs: A Learning Approach to Name-based Bug Detection*, Proceedings of the ACM on Programming Languages (OOPSLA), 2018.

[2] A. Hunt and D. Thomas, *The Pragmatic Programmer: From Journeyman to Master*, Addison-Wesley, 1999.

[3] GitHub, *Using GitHub Copilot code review*, https://docs.github.com/en/copilot/how-tos/use-copilot-agents/request-a-code-review/use-code-review.

[4] CodeRabbit, *CodeRabbit CLI*, https://docs.coderabbit.ai/cli/overview.

[5] Google, *Gemini API Documentation*, https://ai.google.dev/gemini-api/docs.

[6] D. Halter, *Jedi: an autocompletion and static analysis library for Python*, https://github.com/davidhalter/jedi.

[7] TinyDB: A lightweight document-oriented database for Python, https://github.com/msiemens/tinydb.

[8] MutPy: A mutation testing tool for Python, https://github.com/mutpy/mutpy.